

Design Justification Report: A Tiled 128-MAC INT8 Convolution-Reduction Chiplet

ECE 410/510 HW4AI – Milestone 4

Kay Hynes

June 7, 2026

1. Problem and motivation

This project accelerates the multiply-accumulate reduction at the center of a representative 3x3 INT8 convolution from YOLO-nano. The selected layer has a 52x52x64 input, 128 output channels, and a reduction length of $3 \times 3 \times 64 = 576$ for every output pixel and channel. It performs 199,360,512 MACs, or 398,721,024 operations when one multiply and one add are counted separately.

The choice is grounded in the M1 software experiment. In the profiled representative-layer program, `conv3x3_int8_vectorized` consumed 2.275 seconds of 2.379 seconds across 15 calls, or 95.6% of that program’s runtime. The M1 NumPy `im2col-plus-matrix-multiply` implementation ran the layer in a median 160.9 ms and achieved 2.478 GFLOP/s. This is the comparison point required by the project specification. It is not a claim that every complete YOLO-nano implementation spends exactly 95.6% of its time in this one layer; it shows that the selected dense reduction is a suitable, measurable acceleration target.

The final deliverable is a co-processor chiplet for that reduction, not a complete object-detection system and not a complete sliding-window convolution front end. A host supplies activation-reduction elements, loads weights, and accumulates returned tile partials. The chiplet implements the expensive parallel INT8 products and INT32 reductions behind a standard streaming interface. This partition is appropriate for the project goal because it produces a synthesizable, interface-connected accelerator whose implementation tradeoffs and transaction schedule can be measured against the M1 target.

2. Roofline analysis

The M1 **algorithmic arithmetic intensity** counts the input feature map, weights, and final output once:

$$\begin{aligned} & 398,721,024 \text{ FLOPs} / (173,056 + 73,728 + 346,112 \text{ bytes}) \\ & = 672.497 \text{ FLOP/byte} \end{aligned}$$

At this ideal-reuse level the convolution is compute-bound on both the M1 Pro reference roofline and the original proposed accelerator roofline. This analysis motivated a parallel MAC array and INT8 arithmetic: when useful performance is limited by arithmetic throughput, spending area on many small multipliers is more valuable than widening an already sufficient external memory path.

The implemented production chiplet does not achieve the ideal-reuse traffic model. The synthesized weight banks contain 64 entries per lane, so a full 576-element reduction requires nine tiles. Each 64-element tile returns 128 INT32 channel partials over a serialized 64-bit stream. For one layer, the implemented transaction schedule contains 73,728 weight-load beats, 1,557,504 compute-input beats, and 3,115,008 output beats. At eight bytes per beat this is 37,969,920 stream bytes, giving an **implemented-interface arithmetic intensity of 10.501 FLOP/byte**.

This distinction changes the final bottleneck. The mathematical kernel remains compute-bound under ideal on-chip reuse, but the synthesized chiplet is mainly interface/serialization-bound. At the setup-limited projected frequency of 114.536 MHz, one 64-bit stream direction has a rated bandwidth of 0.916 GB/s; the final transaction schedule requires about 0.889 GB/s of aggregate serialized traffic. Figure 1 plots both arithmetic-intensity values and the cycle-measured, timing-projected production point.

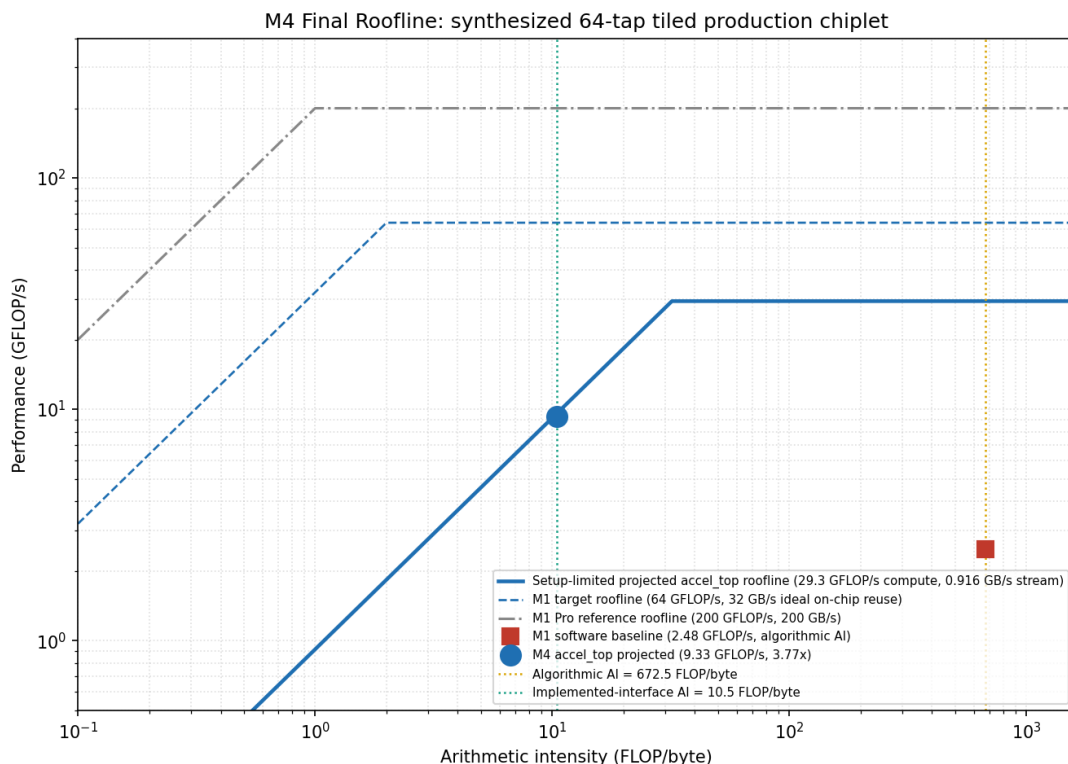


Figure 1: Final roofline. The M1 algorithmic point assumes ideal reuse at 672.5 FLOP/byte. The synthesized tiled chiplet operates at 10.5 FLOP/byte because it transports tile commands and serialized partial sums; combining measured cycles with the setup-limited post-CTS projection gives 9.335 GFLOP/s.

3. Precision and data format

The datapath uses signed INT8 activations and weights with signed INT32 accumulation. Each INT8 multiplication produces a signed 16-bit product, which is sign-extended before accumulation. INT32 is sufficient for a full 576-element reduction: even the conservative magnitude bound $576 \times 16,384$ is below 10 million, well within signed 32-bit range.

INT8 was selected because the target is an edge-oriented dense convolution and because it allows substantially more multipliers per area than FP32. The final precision study tested 1,000 deterministic 576-tap reductions against an FP32 reference, matching the complete reduction reconstructed from nine hardware tiles. It measured mean absolute error 0.0360, RMS error 0.0448, maximum absolute error 0.1503, and relative mean absolute error of 0.564% against the mean reference magnitude. These results establish that the quantized arithmetic is numerically reasonable for the final reduction workload. They do not establish task-level YOLO accuracy; that would require a trained quantized model and a labeled validation set.

Within each lane, the running partial sum is represented in carry-save form as two 32-bit registers. The inner loop computes a bitwise three-input XOR and majority function instead of propagating a carry through a 32-bit adder every cycle. A single carry-propagate addition resolves the result on the tile’s `last` element. Carry-save representation changes timing and area, but it is bit-exact relative to ordinary INT32 addition for the tested reductions.

4. Dataflow and architecture

The compute dataflow is **output-stationary and weight-streaming within each tile**. One activation is broadcast to all 128 lanes each compute cycle. Lane `i` receives the weight for output channel `i` and keeps that channel’s partial sum resident for the 64-element tile. At the end of the tile, all 128 channel partials are captured for serialization. This dataflow fits convolution reduction because all output channels consume the same activation element while using different weights.

The `mac_array` compute engine contains 128 parallel lanes and a three-stage pipeline: Stage A captures activation and weights, Stage B performs the signed 8x8 multiplication, and Stage C updates or resolves the carry-save accumulator. Shared `valid`, `first`, and `last` tags travel with the data. During a tile, the array accepts one reduction element per cycle and performs 128 useful MACs per compute cycle.

The synthesized production boundary is `accel_top`, shown in Figure 2. It adds four necessary chiplet functions:

1. A 64-bit opcode-tagged AXI4-Stream input.
2. 128 private 64-entry-by-8-bit inferred weight banks.
3. A registered activation/control broadcast stage.
4. A result buffer and serializer that returns one channel partial per beat.

The final scope is intentionally a **64-element tiled reduction chiplet**. Nine invocations reconstruct the full 576-element convolution reduction, with the host adding each returned INT32 partial inline as its serialized beat arrives. The final testbench models this one-add-per-output-beat host behavior, so it adds no cycles beyond the already counted drain stream. Host software orchestration, sliding-window generation, padding, and final output conversion outside the stream schedule are not measured. The chiplet does not implement line buffers, sliding-window generation, padding, or final activation quantization. Those functions remain with the host. This is narrower than the original architectural target, but it is the design represented consistently by final simulation, synthesis, and benchmark evidence.

Figure 2. Final production architecture: host-managed nine-tile reduction through synthesized accel_top

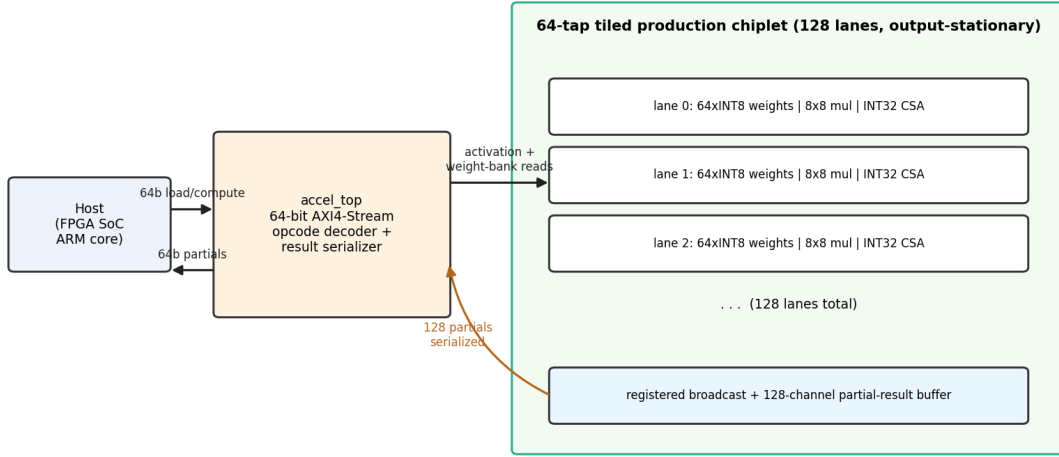


Figure 2: Production architecture. The host loads one 64-weight tile per lane, streams 64 activation elements per output pixel, receives 128 serialized partials, and accumulates nine tiles for the full reduction.

Figure 3. Tiled output-stationary dataflow: 64-element chiplet partials, nine-way host accumulation

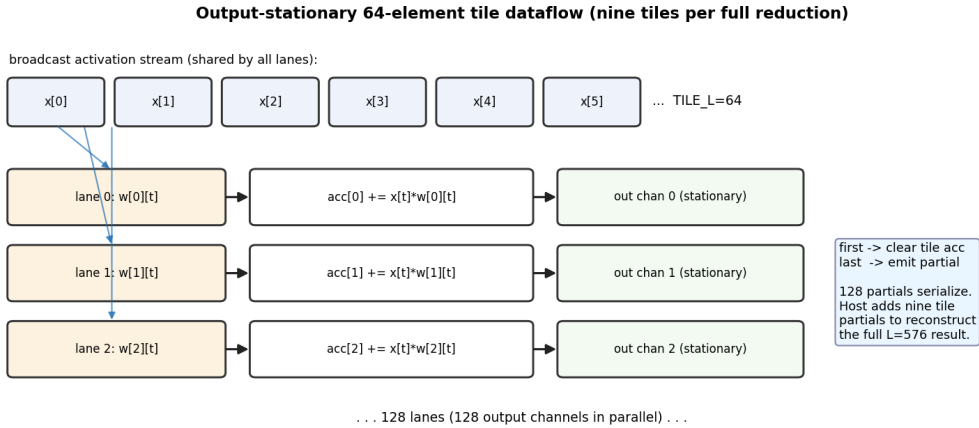


Figure 3: Output-stationary tile dataflow. Partial sums remain in each lane for one 64-element tile; nine returned tile partials are accumulated by the host.

5. Hardware interface

The chiplet exposes a 64-bit AXI4-Stream-style input and output using the standard TVALID, TREADY, and TDATA handshake. AXI4-Stream is appropriate for the assumed FPGA-SoC host because it is a published AMBA streaming protocol, naturally transports ordered commands and results, and could connect to host DMA or control logic without changing the chiplet protocol.

Input opcode 0x01 loads one signed INT8 weight into a selected lane and address. Opcode 0x02 carries a broadcast activation plus `first` and `last` tags for a compute tile. The output returns a seven-bit channel index and one signed INT32 partial sum per beat. The testbench exercises complete weight write, compute, and response transactions exclusively through these ports.

The interface is functionally correct but is the dominant final performance limitation. The serializer requires 128 output beats after every 64 compute beats. `accel_top` has one result buffer and deasserts input TREADY while draining, so the host must wait for a tile result before allowing another tile result to arrive. The production testbench enforces this supported protocol. The result path therefore consumes more cycles than the compute path.

M1’s original bandwidth conclusion assumed a full on-chip 576-entry weight store and ideal feature-map reuse. That architecture would not be interface-bound. The implemented 64-entry tiled design is different, and the final benchmark reports the resulting interface-bound behavior rather than reusing the M1 assumption.

6. Verification

The final required testbench is `tb/tb_top.sv`. It instantiates `accel_top` with the same key parameter used by synthesis, `L_MAX=64`, and drives only the narrow production AXI4-Stream interface. It builds deterministic signed INT8 activations and weights for eight output pixels and computes two independent references: every 64-element tile partial and every full 576-element result. The first pixel and first channel deliberately use alternating 127 and -128 operands, exercising full-scale INT8 inputs and a large-magnitude carry-save accumulation; the remaining vectors provide varied signed values.

For each of nine tiles, the testbench loads 128 x 64 weights, streams each pixel’s 64 activation elements, receives 128 serialized channel partials, and adds those partials into a host-side accumulator. It checks 9,216 tile partials and 1,024 reconstructed full results. The committed final log reports:

```
partial_errors=0 full_errors=0
total_cycles=87988
backpressure_cycles=3 backpressure_errors=0 unstalled_schedule_cycles=87985
useful_macs_per_total_cycle=6.703
PASS: final synthesized-config accel_top tiled 576-element reduction
```

The test deliberately deasserts output TREADY for three cycles and still passes every ordering and value check. The 87,985-cycle unstalled schedule, rather than the injected protocol-test stall, is used for layer extrapolation. The low MAC/cycle value in this short verification run includes loading all 73,728 weights but amortizes them over only eight pixels. The full-layer benchmark amortizes the same weight load over 2,704 pixels. More importantly, the test validates the exact transaction categories used by the benchmark: weight loads, compute beats, serialized drains, and protocol gaps. Figure 4 shows a compute tile entering the array and the resulting partial entering

the serializer.

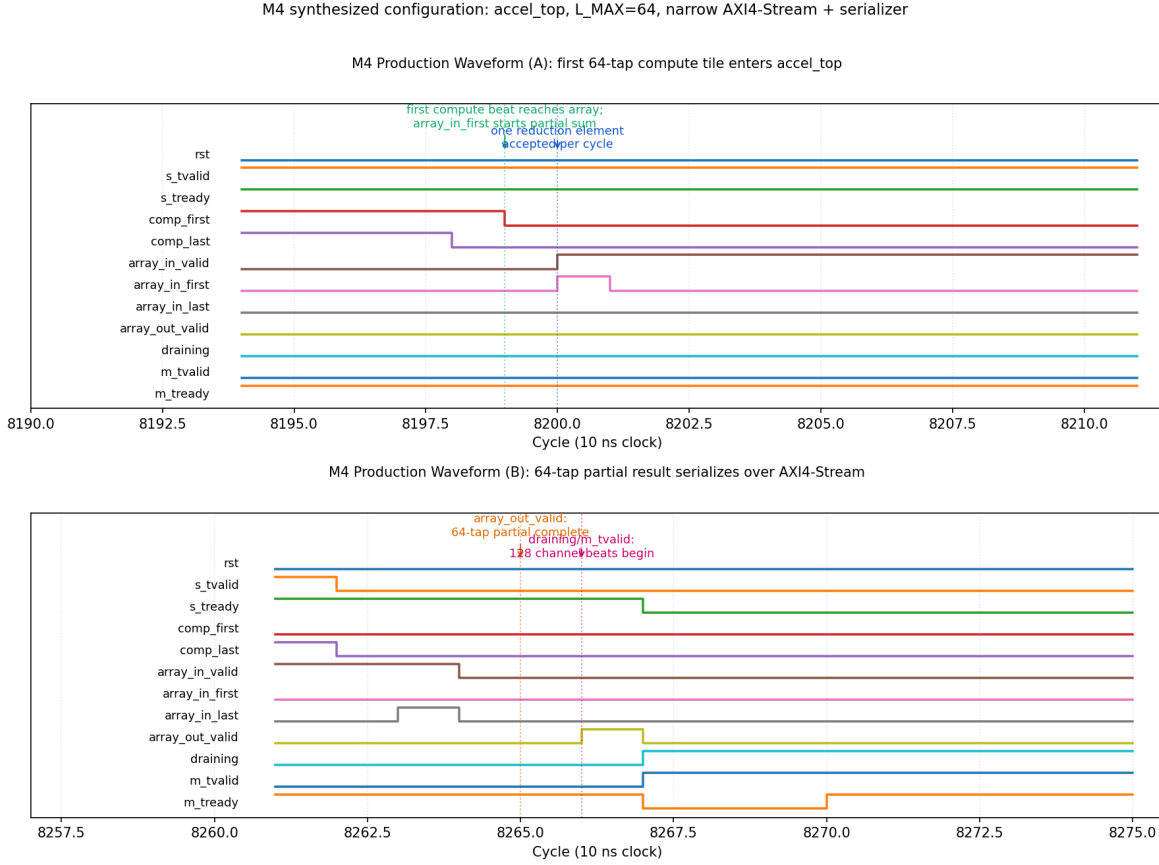


Figure 4: Annotated production waveform showing a 64-tap tile entering accel_top and its 128-channel partial result beginning serialized output.

Earlier M2 and M3 tests remain useful developmental evidence for the individual MAC arithmetic and initial interface integration. The final M4 pass condition, however, is based on the synthesized production configuration.

7. Synthesis results

Synthesis used OpenLane 2 v2.3.10 with the sky130A sky130_fd_sc_hd standard-cell library. The final production configuration is `synth/config.json`: accel_top, 128 lanes, L_MAX=64, a 6.0 ns target, and a 3.3 mm by 3.3 mm die. This is the configuration matched by final verification and benchmarking.

The full production wrapper completed synthesis, floorplanning, placement, clock-tree synthesis, post-CTS timing repair, and global routing. The deepest available snapshot contains 558,792 standard cells and 5.015 mm² of standard-cell area. The dominant contributors are the inferred weight-register banks, 128 multipliers and carry-save accumulators, clock tree, and timing-repair buffers. The 65,536 required weight bits alone account for at least 77.17% of the design's 84,927 sequential cells; the 70,646 timing-repair buffers account for 12.64% of all post-CTS cells. Global routing reached zero overflow on every metal layer.

At the post-CTS typical corner, the 6.0 ns target has setup WNS of -2.730896 ns. The resulting setup-limited projected period is 8.730896 ns, or **114.536 MHz**. This full-wrapper number is used in the final benchmark. The same snapshot has -1.852060 ns hold WNS and 32,070 reported hold violations, although register-to-register hold worst slack is positive 0.197419 ns with zero register-to-register hold violations. It also has two max-slew violations and one max-capacitance violation. Detailed routing and final timing repair did not complete, so these remain unresolved; 114.536 MHz is a setup-limited estimate, not sign-off timing, and no slow-corner full-wrapper frequency is claimed.

The full-wrapper worst setup path starts at a result-path flip-flop, passes through fanout buffers and serializer mux/output logic, and ends at `m_tdata[26]`, with -2.730896 ns slack. The production interface, rather than the per-lane multiplier, is therefore the final full-wrapper timing limiter. The run also used OpenLane’s generic fallback SDC because no explicit `PNR_SDC_FILE` or `SIGNOFF_SDC_FILE` was supplied. A follow-on run must define host-specific input/output delays, complete detailed routing, and rerun setup/hold repair before claiming interface timing closure.

The full-wrapper post-CTS power estimate is **1.018 W** at the typical corner with default switching activity. Sequential logic and clock distribution dominate. This estimate is pre-detailed-route and not activity-annotated, so it is used only as an approximate energy result. It was analyzed under the 6.0 ns target constraint and is not scaled to the slower setup-limited frequency.

8. Benchmark results

The final benchmark derives full-layer cycles from the transaction schedule measured by `tb_top.sv`. One layer requires:

```
73,728 weight-load cycles
1,557,504 compute-input cycles
3,115,008 serialized-output cycles
146,017 pipeline and protocol-gap cycles
= 4,892,257 total cycles
```

This is **40.750 useful MACs per total chiplet cycle**. Combining that measured cycle schedule with the setup-limited full-wrapper post-CTS projection of 114.536 MHz gives 42.714 ms and **9.335 GFLOP/s**. Against the required M1 median of 160.9 ms and 2.478 GFLOP/s, this is a **3.77x projected speedup** for the same useful convolution FLOP count. However, it is an optimistic chiplet-schedule comparison: M1 includes NumPy `im2col/padding` and output processing, while M4 excludes host orchestration, sliding-window generation, padding, and final output conversion. Because of that scope difference and because the full wrapper does not close timing, this is not a demonstrated end-to-end operating point.

Combining the full-wrapper power estimate with the projected runtime gives approximately 43.483 mJ per layer and 9.17 GFLOP/s/W. This is an arithmetic estimate, not a demonstrated operating point. No software energy improvement ratio is claimed because the M1 CPU baseline did not include a measured power value.

The original theoretical target was 64 GFLOP/s at 250 MHz. The final gap is not caused by arithmetic correctness. It comes from the lower full-wrapper frequency projection, nine-way tiling required by the synthesized weight capacity, and serialized tile-partial output. These observed limitations are reflected in the final roofline.

9. What did not work

The first major failure was scope: a full 576-entry weight bank for every lane was too large when inferred as flip-flops in the available educational flow. The final synthesizable production configuration therefore uses 64 entries and host-managed tiling. A better implementation would integrate SRAM macros so the complete 576-element weights remain on chip.

The second failure was treating the compute-array rate as accelerator throughput. The wide unit-test wrapper and the inner compute phase can sustain nearly 128 MACs/cycle, but the production wrapper must serialize 128 channel partials and has only one result buffer. When the final testbench was changed to 64-tap tiles, issuing pixels without waiting for serialization overwrote the result buffer. The corrected host protocol waits for each result drain, and the benchmark includes that cost. A next version should use double buffering, a FIFO, or a wider output stream to overlap compute and drain.

The attempted eight-bank broadcast optimization also did not work as intended. `accel_top` creates eight registered activation/control copies, but `mac_array` exposes only one scalar shared activation/control input, so only bank 0 is connected and the unused copies are eligible for synthesis removal. The final design therefore has registered broadcast staging, not a true eight-bank fan-out tree. A real banking fix requires splitting `mac_array` into banked instances or changing its interface so each lane group consumes its own registered copy.

The third major failure was timing closure. In the separate routed-lane experiment, carry-save accumulation moved the critical path away from the 32-bit accumulation adder and exposed the 8x8 signed multiplier as the new lane limiter. The full wrapper was slower still because of its weight storage, shared broadcast network, clock distribution, and routing. It also retains unresolved non-register-to-register hold, slew, and capacitance violations at the final snapshot. Pipelining the multiplier would improve the lane, while completed timing repair and hierarchical physical design would better address the full array. I would also provide an explicit host-interface SDC instead of the generic fallback so input/output timing and the serializer critical path are constrained against the assumed FPGA-SoC.

The fourth failure was full detailed routing. The bare array first failed because exposing all wide weights and results created 5,134 top-level pins. `accel_top` reduced that to 137 pins and completed global routing with zero overflow, but TritonRoute failed during detailed-routing track assignment on the available 16 GB host. The committed `openlane_run.log` records this failure, and the report uses only the deepest available full-wrapper post-CTS results.

Finally, power estimation is not workload-annotated. The 1.018 W result uses default activity and lacks final routed parasitics. A stronger follow-on would complete detailed routing and replay the production testbench with VCD or SAIF activity.

The final design is narrower and slower than proposed in M1, but it is a coherent engineering result: the same 64-tap production chiplet is verified, synthesized, analyzed, and benchmarked; it correctly reconstructs the target 576-element reduction. Its cycle schedule supports a projected kernel advantage, but timing closure and an end-to-end host benchmark remain required before claiming demonstrated acceleration.